Explanation
The first statement asks the user to enter the first value. The value entered by the user is stored in val1 (2678). The statement `print('val1 =', val1, 'type:', type(val1))` displays the output on the screen as: val1 = 2678 type: <class 'int'>. It can be observed that val1 is written as it is, since it is contained inside the single quote. After the comma, val1 is written without quotes, hence the value of the identifier val1 will be displayed (2678). After the comma, type is written in single quotes, hence it is printed as it is and then type(val1) is printed. Since the user entered an integer value, class is printed as int. Similarly in the next example, the user enters a float value (1398.26); hence the class is printed as float. In the last example, the user enters a string in double quote; hence the class is printed as "str" that represents the string.

In Python, unlike other programming languages like C, C++, it is possible to give multiple inputs by the user using one single input statement. This helps in assigning multiple values to multiple variables in one statement. Example: `var1, var2 = eval(input('Please enter number 1 and number 2: '))` will prompt user to enter two numbers var1 and var2.

```
#Program to show use of accepting multiple inputs using eval() function.
a,b,c = eval(input('Enter three numbers: '))
print(a,'+',b,'+',c, '=', a+b+c)
```

```
In [1]: runfile(...)
Enter three numbers: 10, 15, 6
10+15+6 = 31
```

Explanation
This example shows that the user is able to enter three numbers at the same time using one `eval()` function. The user has entered three numbers 10, 15, and 6 and hence the result is 31. It should be noted that for giving multiple inputs through one statement, the numbers entered by the user should also be separated by a comma as shown in the `eval()` function.

The statement `a,b,c= int(input("Enter three numbers:"))` will result in run-time error because `int()` function cannot accept multiple inputs. This is possible only through `eval()` function.

1.7 Operators

An operator is a symbol that tells the compiler to perform specific mathematical or logical functions. Python language provides the following types of operators: arithmetic operators, assignment operators, relational operators, logical operators, and Boolean operators. This section discusses these operators in detail.

1.7.1 Arithmetic Operators

There are different arithmetic operators like Addition (+) for adding operands; Subtraction (–) for subtracting second operand from first; Multiplication (*) for multiplying operands; Division (/) for dividing numerator by denominator; Modulus operator (%) for determining the remainder after an integer division; Exponential operator (**) for calculating exponential power value; and Integer division (//) for performing integer division and displaying only integer quotient. It should be noted that in an expression where multiple operations take place, the bracket (parenthesis) will be given the priority, followed by exponential, then division, multiplication, modulus, addition and subtraction in this order only. The assignment operator is given the last priority and hence is at the end; the value of right-hand side of expression is stored in the left-hand side of the expression.

```
#Program to show the usage of arithmetic operators.
num1 = int(input('Enter first number: '))
num2 = int(input('Enter second number: '))

#Using (+) to add two integers provided by the user.
print(num1, '+', num2,     '=',num1+num2)

#Using (-) to subtract two integers provided by the user.
print(num1, '-', num2,     '=',num1-num2)

#Using (*) to multiply two integers provided by the user.
print(num1, '*', num2,     '=',num1*num2)

#Using (/) to divide two integers provided by the user.
print(num1, '/', num2,     '=',num1/num2)

#Using (%) to determine the remainder from two integers.
print(num1, '%', num2,'=',num1%num2)

#Using (**) to determine exponential power from two integers.
print(num1, '**', num2,'=',num1**num2)

#Using (//) to perform integer division on two integers.
print(num1, '//', num2,    '=',num1//num2)
```

```
In [1]: runfile(...)
Enter first number: 101
Enter second number: 4
101 + 4 = 105
101 - 4 = 97
101 * 4 = 404
101 / 4 = 25.25
101 % 4 = 1
101 ** 4 = 104060401
101 // 4 = 25
```

Explanation

The first two statements prompt the user to enter two numbers. When the user types the number 101 and then presses the enter key, num1 is assigned to the integer 101. Similarly, the user enters number 4, which is stored in num2. Later, the different arithmetic operators produce the desired result. It can be observed that 101/5 gives a float result, while 101//5 performs integer division and hence gives only the integer result and ignores the decimal part. The remainder 1 is displayed using modulus operator (%). The ** is used for exponential computation.

1.7.2 Assignment Operators

The different types of operators include assignment operator (=) that assigns numbers from right-side operands to left-side operands. For example, Profit = Amount - Expenses will assign the value of Amount - Expenses to Profit. It should be noted that unlike C and Java, Python does not have increment and decrement operators. The following example demonstrates the use of different assignment operations in Python.

```
#Program to use assignment operators.
num1 = int(input('Enter first number: '))
num2 = int(input('Enter second number: '))

#Add AND assignment operator(+=): It adds right to left operand and assigns the result to the
#left operand. Y += X is equivalent to Y = Y + X.
result=num2
result+=num1
print("The result of addition assignment is: ", result)

#Subtract AND assignment operator(-=). It subtracts right from left operand and assigns
#result to left operand. Y-= X is equivalent to Y= Y-X.
result=num2
result-=num1
print("The result of subtraction assignment is: ", result)

#Multiply AND assignment operator (*=). It multiplies right with left operand and assigns
#result to left operand. Y*= X is equivalent to Y= Y*X.
result=num2
result*=num1
print("The result of multiplication assignment is: ", result)

#Divide AND assignment operator (/=). It divides left with right operand and assigns the result
#to left operand. Y /= X is equivalent to Y = Y / X.
result=num2
result/=num1
print("The result of division assignment is: ", result)

#Integer Division AND assignment operator(//=). It divides left with right operand and
#assigns the integer quotient to the left operand.
result=num2
result//=num1
print("The result of integer division assignment is: ", result)

#Modulus AND assignment operator (%=). It divides left with right operand and assigns
#remainder to the operand. Y %=X is equivalent to Y= Y%X.
result=num2
result%=num1
print("The result of modulus assignment is: ", result)

#Exponential AND assignment operator (**=) Divides left with right operand and assigns
#remainder to left operand. Y**= X is equivalent to Y= Y**X.
result=num2
result**=num1
print("The result of exponential assignment is: ", result)
```

```
In [1]: runfile(...)
Enter first number: 2
Enter second number: 51
The result of addition assignment is: 53
The result of subtraction assignment is: 49
The result of multiplication assignment is: 102
The result of division assignment is: 25.5
The result of integer division assignment is: 25
The result of modulus assignment is: 1
The result of exponential assignment is: 2601
```

QUICK CHALLENGE

For the above input, print the result as The result of basic mathematical operations on 2 and 51 is 53, 49, 102 and 25.

1.7.3 Relational Operators

The different relational operators supported in Python are Equals (==), not Equals (!=), Greater than (>), Less than (<), Greater than or equal to (>=), Less than or equal to (<=). The use of these operators is explained in the following program:

```
#Program to use relational operators in a program.
num1=60
num2=30

#The Equals(==) operator checks if the value of two operands are equal.
print("The result of equals to operator is: ",num1==num2)

#The Not equals(!=) operator checks if value of two operands are not equal.
print("The result of not equals to operator is: ",num1!=num2)

#The Greater than (>) operator checks if value of left operand is greater than right operand.
print("The result of greater than operator is: ",num1>num2)

#The (>=) operator checks if value of left operand is greater than or equal to right operand.
print("The result of greater than or equals to operator is: ",num1>=num2)

#The less than (<) operator checks if value of left operand is less than right operand.
print("The result of less than operator is: ",num1<num2)

#The <= operator checks if value of left operand is less than or equal to right operand.
print("The result of less than or equals to operator is: ",num1<=num2)
```

```
In [1]: runfile(...)
The result of equals to operator is: False
The result of not equals to operator is: True
The result of greater than operator is: True
The result of greater than or equals to operator is: True
The result of less than to operator is: False
The result of less than or equals to operator is: False
```

1.7.4 Logical Operators

These operators are used when we need to check multiple conditions. Each of the condition is evaluated to True or False and then the combined decision related to all conditions is taken. There are basically three logical operators in Python which include "and", "or", and "not". The "and" operator returns True if all the conditions are True. The "or" operator returns True if at least one of the conditions is True. The "not" operator returns the opposite of the value. This means that if value is "True", it will return "False" and vice versa. These operators are explained in the following program:

```
#Program to show the use of logical operators.
num1= 250
num2= 350
num3= 275

#Use of "and" logical operator.
print("The use of and operator: ",num1>num3 and num2>num3)

#Use of "or" logical operator.
print("The use of or operator: ",num1>num3 or num2>num3)

#Use of "not" logical operator.
print("The use of not operator: ", not num1<num3)
```

```
In [1]: runfile(...)
The use of and operator: False
The use of or operator: True
The use of not operator: False
```

Explanation
In the first line with print command, num1>num3 does not hold good and num2>num3 holds correct. Since all the conditions are not holding correct for the "and" operator, hence the result is "False". In the second line with print command, result is "True" because one condition is holding correct. In the last line, num1<num3 is "True", but when "not" operator is used, we get the result as "False".

It is also possible to use logical operator on Boolean values. The Boolean input will always return a Boolean output as shown in the following program:

```
#Program to show the use of logical operators on Boolean values.
val1= True
val2= False

#Use of "and" on Boolean values.
print("The use of and operator returns: ", val1 and val2)
print("The use of and operator on val1 only returns: ", val1 and val1)
print("The use of and operator on val2 only returns: ", val2 and val2)

#Use of "or" on Boolean values.
print("The use of or operator returns: ", val1 or val2)
print("The use of or operator returns: ", val2 or val1)

#Use of "not" on Boolean values.
print("The use of not operator on val1 returns: ", not val1)
print("The use of not operator on val2 returns: ", not val2)
```

```
In [1]: runfile(...)
The use of and operator returns: False
The use of and operator on val1 only returns: True
The use of and operator on val2 only returns: False
The use of or operator returns: True
The use of or operator returns: True
The use of not operator on val1 returns: False
The use of not operator on val2 returns: True
```

Explanation
The use of "and" operation on val1 and val2 returns "False" since both are not True. However, "val1 and val1" returns "True" since both the conditions are True. Similarly, "val2 and val2" returns "False" since both the conditions are False. The use of "or" operator returns "True" since at least one of the conditions is True. The "not val1" returns False (opposite of val1 that is True) and "not val2" returns True (opposite of val2 that is False).

1.7.5 Operator Precedence

In an expression with multiple operators, operator precedence determines the grouping of terms in an expression and decides how an expression will be evaluated. Certain operators have higher precedence than others. For example, ans = 10 + 5 * 4; here, ans is assigned 30, not 60. Since the multiplication operator has higher precedence than the addition operator, so 5 is first multiplied with 4 and then added to 10. The following table shows operator precedence in descending order:

Operator	Details
()	Parenthesis
**	Exponential
/, *, //, %	Division, Multiplication, Integer Division, Modulus
+, −	Addition and Subtraction
>, >=, <, <=, ==, !=	Relational Operators
=, +=, −=, **=, /=, //−, %=	Assignment Operators
not, or, and	Logical and Boolean operators

Explanation
In this program, the user is given three choices to determine whether the string is pangram, consogram or vowgram. A string named letters is created corresponding to the choice of the user. If the user enters pangram, letters contains all the English alphabets, if the user enters consogram, letters contains all the consonants and if the user enters vowgram, letters contains all the vowels. A Boolean variable named flag is created with default value as True. Each and every letter from the letters is checked in user string. If any letter is not found in the string, the Boolean variable will be assigned the value as false and the loop will exit with the break statement (refer to Section 2.4.1 for detail). If flag is True, this means all the letters were found in the string, hence it is a pangram/consogram/vowgram.

2.2 Loops

A loop statement allows us to execute a statement or group of statements multiple times. Repetitive commands are executed by loops: for loop, while loop, and repeat loop. Python loops are particularly flexible and they are not limited to integers. We can also use loops for character vectors, logical vectors, lists, or expressions.

2.2.1 For Loop

If the number of repetitions are known in advance, then a "for" loop can be used. For executing for loop, we need to specify start, end, and step argument. The command inside the for loop is executed from the counter having specified value as start; every time it is incremented by the value which is specified in step; the new value of the counter is checked till it reaches the value specified in the end argument (Fig. 2.5).

Syntax

```
for var in range(start, end, step):
    commands to be executed
```

where

- `var` is the variable name
- `start` denotes the starting number
- `end` denotes the ending number
- `step` is the optional step for increment or decrement (default are +1 and −1)

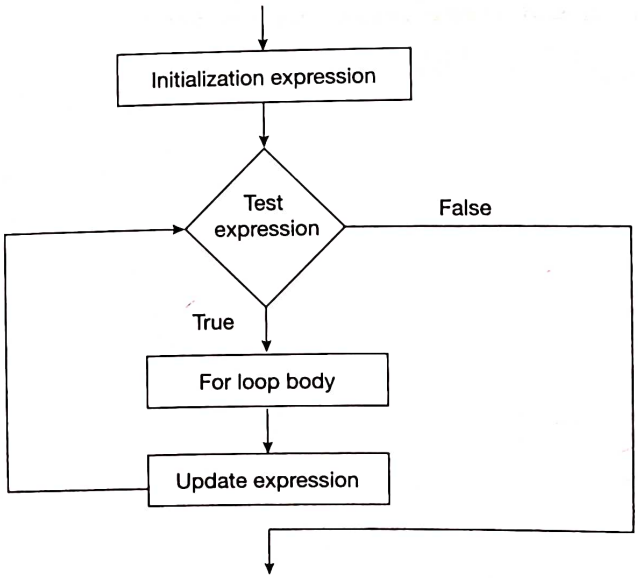

Figure 2.5 Flowchart of "for" iterator.

```
#Program for printing the table of a number.
num=int(input("Input the number to print the table: "))
for k in range(1,11):
    print(num,"x", k,"=",num*k)

#Outside the for loop.
print("Finished Task")
```

```
In [1]: runfile(...)
Input the number to print the table: 7
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70
Finished Task
```

Explanation

The user is prompted to give input of a number which is stored in "num". The for loop uses a counter named "k" and has two arguments for range: 1 and 11. This implies the starting value of "k" will be 1 and the last value will be less than 11, which means that the last value will be 10. The most important thing to observe is that the range function gives us values up to, but not including, the upper limit. The absence of third-step argument implies that the default increment of 1 will be applied. This means that the value of "k" will be incremented by 1 each time till it reaches the value 11. This further implies that initially for "k" equal to 1, the print statement will be executed; the value of "k" will then be incremented by 1 (k=2) and again the print statement will be executed. This process will be continued till "k" is equal to 10. When the value of "k" reaches to 11, the loop will stop getting executed and the control will pass to the main program and hence "Finished Task" is printed.

However, the same program with different specified step argument will show different results as discussed in the following section.

```
#Program for printing the table of a number using step argument.
num=int(input("Input the number to print the table: "))
for k in range(1,11,2):
    print(num,"x", k,"=",num*k)

#Outside the for loop.
print("Finished Task")
```

```
In [1]: runfile(...)
Input the number to print the table: 4
4 x 1 = 4
4 x 3 = 12
4 x 5 = 20
4 x 7 = 28
4 x 9 = 36
Finished Task
```

Explanation

The third argument in range is 2. This implies that the step argument is 2 which further means that each time, the value of "k" will be incremented by 2, unlike the earlier case where value of "k" was incremented by default value of 1. Hence the table of 4 is printed only for k = 1, 3, 5, 7, and 9.

It is also possible to have negative value for decrementing the value of step. For example, if we need to print the table in decreasing order, we should have syntax as: for k in range(10, 0, -2). In this case, the loop will start from 10 and continue till 1 decrementing the value by 2 each time.

It is also possible to accept the number from the user for defining the starting and ending arguments. This is shown in the following example:

```
#Program to print sum of the series 1+2+3+4.....
sum=0
num=int(input("Input the last number of series: "))
for k in range (1,num+1):
    sum=sum+k

#Outside the "for" loop.
print("The total is:", sum)
print("Finished Task")
```

```
In [1]: runfile(...)
Input the last number of series: 5
The total is: 25
Finished Task

In [2]: runfile(...)
Input the last number of series: 9
The total is: 45
Finished Task
```

Explanation

For printing the sum of the series starting from 1 till a number, the user is prompted to enter the number which is stored in "num". The starting value of "sum" is 0. The "for" loop is executed for "num" times, since the starting value of "k" is 1 and the ending value is "num+1". Each time the value of "k" is incremented by default value of 1.

The following example reads a number from the user. Find the difference between the sum of squares up to that number and the number itself.

```
#Program to calculate difference between sum of squares and the number.
number=int(input("Enter the number: "))
sum =0
for i in range(1,number+1):
    sum = sum + (i *i)

#Outside the for loop.
ans = sum - number
print("The answer is",ans)
```

```
In [1]: runfile(...)
Enter the number: 3
The answer is 11

In [2]: runfile(...)
Enter the number: 4
The answer is 26
```

Explanation

The number entered by the user is stored in the variable named "number". The range inside the for loop is specified from 1 to number+1; this will execute for loop till the number. The variable "sum" thus stores the sum of the squares from 1 till that number. The variable "ans" will deduct number from sum. For example: if a number is 3, then the output will be 11 (i.e., 1 *1 + 2 *2 + 3 *3 - 3 = 11).

We can also have different starting value. This is shown in the following program where the starting value of the counter is other than 1.

```
#Program to generate Fibonacci series.
num=int(input("Numbers of Fibonacci series to be printed: "))
a=0
b=1
print(a)
print(b)

#Executing for loop.
for k in range(3,num+1):
    c=a+b
    a=b
    b=c
    print(c)

#Outside the for loop.
print("Finished Task")
```

```
In [1]: runfile(...)
Numbers of Fibonacci series to be printed: 6
0
1
1
2
3
5
Finished Task
```

Explanation

The first two numbers of Fibonacci series are 0 and 1 and are printed before "for" loop. Hence, the starting value of counter "k" is 3 and the loop will execute till y+1. The "for" loop has four statements inside it, hence all the four statements are executed every time the value of "k" is incremented. Since in the Fibonacci series all the numbers need to be printed, hence the print statement for printing the number is kept inside "for" loop. However, since "Finished Task" is outside for loop, hence "Finished Task" is printed when the control is transferred outside for loop to the main program.

The utility of "for" loop can be also shown in another example of calculating factorial of a number.

```
#Program for calculating factorial of a number.
num=int(input("Input the number:  "))
fact=1
for k in range(1,num+1):
    fact=fact*k

#Outside the for loop.
print("The answer is:", fact)
print("Finished Task")
```

```
In [1]: runfile(...)
Input the number:   6
The answer is: 720
Finished Task

In [2]: runfile(...)
Input the number: 8
The answer is: 40320
Finished Task
```

Explanation

The user is prompted to enter a number for calculating the factorial which is stored in num. The value of "fact" is 1. The co
"k" has initial value as 1 and will execute till the value of "k" reaches num+1. The control will be transferred back to the mai
gram when the value of counter "k" reaches to num+1. The program shows that for num=6, the initial value of "k" is 1
will execute the statements written inside "for" till "num" reaches the value 6. For the first time when fact=1, k = 1, "f

will be equal to 1 (1*1). The second time when the loop got executed, the initial value of "fact" is 1 and k = 2 and "fact" will be equal to 2 (2*1). The third time the loop got executed, initial value of "fact" is 2 and k = 3 and "fact" will be equal to 6 (2*3). Similarly, it will execute till "k" reaches the value 6 and hence final value of "fact" will be 720. It can be observed that the value of "fact" is printed not inside the "for" loop and hence the final value of "fact" (720) is printed. Using the same approach, the factorial of 8 is calculated and the final "fact" is printed.

Display the values of each computation. Like in the program in the previous page, it should display 6 * 1 = 6, 6 * 2 * 1 = 12, 6 * 3 * 2 * 1 = 36, 6 * 4 * 3 * 2 * 1 = 144, 6 * 5 * 4 * 3 * 2 * 1 = 720. It should also be printed in descending order: 6 * 5 * 4 * 3 * 2 * 1 = 720, 6 * 4 * 3 * 2 * 1 = 144, 6 * 3 * 2 * 1 = 36, 6 * 2 * 1 = 12, 6 * 1 = 6.

Since space has a lot of importance in Python, hence proper care should be taken while writing a statement. The following program shows the difference in output when the print(fact) is considered inside the for loop.

```
#Program to show factorial of a number.
x=int(input("Input the number :  "))
fact=1
for k in range(1,x+1):
    fact=fact*k
    print("The answer is:", fact)

#Outside the for loop.
print("Finished Task")
```

```
In [1]: runfile(...)
Input the number:  5
The answer is: 1
The answer is: 2
The answer is: 6
The answer is: 24
The answer is: 120
Finished Task
```

Explanation

In this program, the value of fact is printed inside the "for" loop, hence before incrementing the value of counter "k" each time, the print statement is executed and hence the "fact" is printed multiple times. The user accordingly use spaces.

In real-world scenarios, with the increasing complexity of program, there is a need to include loops and conditional program in the same program as shown in the following example:

```
#Program to show use of "For" and "If" in the same program.
num=int(input("Input the number of Items: "))
amount=0
discount=0

#Using for loop.
for k in range(1,num+1):
    price=eval(input("Input the price: "))
    quantity=eval(input("Input the quantity: "))
    amount=amount+(price*Quantity)
print("Total is :",amount)

#Using "if" conditional statement.
if amount>5000:
    discount=amount*0.1
    print("Discount of 10% = ", discount)
elif amount>3000:
    discount=amount*.07
    print("Discount of 7% = ", discount)
```

```
elif amount>1000:
    discount=amount*.05
    print("Discount of 5% = ", discount)
else:
    print("No discount received")

#In main program.
print("Net amount is: ",amount-discount)
print("Finished Task")
```

```
In [1]: runfile(...)
Input the number of Items: 2
Input the price: 200
Input the quantity: 4
Input the price: 100
Input the quantity: 6
Total is: 1400
Discount of 7% = 98
Net amount is: 1302
Finished Task
```

Explanation

This example calculates the net payable amount in the shop depending on the bill amount. If the bill amount is greater than 5000, 10% discount is provided; if it is between 3001 and 5000, 7% discount is provided; and if it is between 1001 and 2999, 5% discount; else no discount is provided. The number of products are entered by the user and stored in "num". The "for" loop is executed "num" times and each time the price and quantity are entered by the user. The amount is calculated and discount is provided according to predefined conditions using conditional statement.

2.2.2 Nesting of `for` Loops

A loop contained within another loop is called a *nested loop*. These are used when an iterative process is required. These are generally used in multi-dimensional or hierarchical data statements, including matrices, lists, etc. It is important to understand that we can nest "for" loops to a great level, although nesting beyond 4 to 5 levels often makes it difficult to read and understand the code.


STUDY HINT

In nested "for" loops, it is important to emphasize on the spaces left before another nested loop. The reader should write the statements carefully with proper space indentation.

```
#Program to print the number of stars.
num=int(input("Input the number of lines: "))
for k in range(1,num+1):
    for j in range (1,k+1):
        print("*",end=" ")
    print("\n")

#Outside the for loop.
print("Finished Task")
```

```
In [1]: runfile(...)
Input the number of lines: 6
*
* *
* * *
* * * *
* * * * *
* * * * * *
Finished Task
```

Explanation

We wanted to print stars in number of lines, and the number of stars in a particular line depends on the value of "num". Thi implies that the first line will have one star, the second line will have two stars, and so on. This means that we need one counte

to represent line number and one counter to represent the number of stars, which further means that we require two counters and hence two "for loops". The user is prompted to enter the number of lines which is stored in "num". The first for loop starts from 1 and will continue till "num". Since we have "for" loop inside "for" loop, hence, for every increment of "k", the other "for" loop will start in which the counter "j" will have values from 1 to "k". This implies that when k = 1 , "j" will have values from 1 to 1; when k = 2, "j" will have values from 1 to 2; when k = 3, "j" will have values from 1 to 3, ... Thus, when the user enters 6, 6 lines are printed with different number of stars.

Create the program as in the previous page using the nested "while" loops and create appropriate conditions for correct results.

2.2.3 While Loop

The "while" loop executes the same code again and again until a stop condition is met. A condition is tested in starting of "while" loop. If it is true, then loop body is executed. Once the loop body is executed, the condition is tested again, and so forth, until the condition is false, after which the loop exits. This means that when the number of loops is not known in advance, we use a "while" loop. A situation that might occur in a "while" loop is that the loop might not run ever. When the condition is tested and the result is false, the loop body will be skipped and the first statement after the while loop will be executed (Fig. 2.6). Besides, the programmer needs to be careful that the counting variable within the loop is incremented, otherwise an infinite loop occurs.

Syntax

```
while (condition):
        commands to be executed as long as condition is TRUE
```

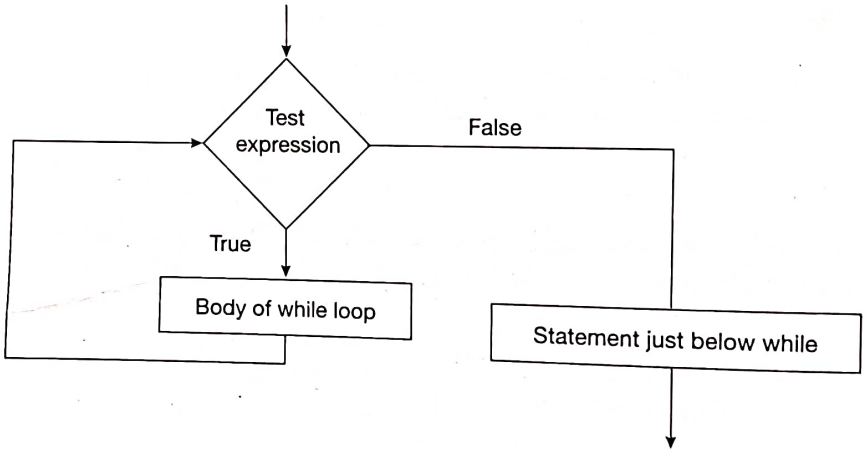

Figure 2.6 Flowchart of "while" loop.

```
#Program using while loop to print numbers within a range.
counter = 101
while counter <= 105:
    print(counter)
    counter += 1

#Outside the while loop.
print("Finished Task")
```

```
In [1]: runfile(...)
101
102
103
104
105
Finished Task
```

Explanation

In this example, value of counter is incremented by 1 inside the "while" loop, hence each time the loop gets executed, the value increases by 1 which further means that at one stage when the value of counter exceeds the value 105, the loop gets terminated and ends in successful end of the program.

It is important to note that the value of the counter within the "while" loop should be incremented, otherwise an infinite loop may occur. This is demonstrated in the following example:

```
#Incorrect program using while (in absence of counter increment).
counter = 101
while counter <= 105:
    print(counter)

#Outside the while loop.
print("Finished Task")
```

```
In [1]: runfile(...)
101
101
```

Explanation

In this program, the initial value of counter was equal to 101 and the loop was supposed to execute till `counter <= 105;` we display the value of the counter. We can observe that since the value of count is not changed and it remains the same (`counter = 101`), value of counter (`101`) is displayed infinite times and the loop is termed as an infinite loop. In this scenario, last statement `Finished Task` is never printed.

The loops used in this example are known as definite loops. In definite loops, the programmer knows the number of times the loop is executed in advance. We know that the definite loops are implemented using "for" loop also. But, in some real-world situations, a programmer cannot always predict how many times a loop will execute. In these situations, indefinite loop is required and the while iterator is ideal for indefinite loops.

We will consider the same program discussed in "for" loop for calculating the amount of the bill. However, the difference is that the program will execute according to user requirement. In real world, the user does not know the number of products in advance. Since the number of products is not known in advance, we will consider a "while" loop which is ideal for indefinite loops.

```
#Program to show use of while loop for computing bill amount.
amount=0
choice="yes"

#Using while loop.
while (choice=="yes"):
    price=eval(input("Input the price: "))
    quantity=eval(input("Input the quantity: "))
    amount=(price*quantity)+amount
    choice=input("Wish to Continue- yes/no: ")

#Outside the loop.
print("Your bill amount is:", amount)
print("Finished Task")
```

```
In [1]: runfile(...)
Input the price: 300
Input the quantity: 2
Wish to Continue- yes/no: yes
Input the price: 200
Input the quantity: 1
Wish to Continue- yes/no: no
Your bill amount is: 800
Finished Task
```

Explanation
In this program, the user is not asked about the number of products in advance and the loop is executed till the user enters
Once the user enters "no", the control is transferred to the main program.

```
#Program to find the sum of digits of a number.
num=int(input("Enter the number: "))
sum=0

#Using while loop.
while (num>0):
    rem=num % 10
    sum=sum +rem
    num = num//10

#Outside the loop.
print("The sum of the numbers is:",sum)
```

```
In [1]: runfile(...)
Enter the number: 567
The sum of the numbers is: 18

In [2]: runfile(...)
Enter the number: 4281
The sum of the numbers is: 15
```

Explanation
This program calculates the sum of digits of a number. For example: if number is 564, then sum = 5 + 6 + 4 = 15. The while loop is used to execute a repetitive task, because the number of times the loop will get executed is not known in advance and varies according to the number entered by the user, which the user does not know. Each time the loop is executed, the mod is determined by using the operator %. This will store the remainder in rem. Thus, when the number 567 is entered, the first time the loop is executed, the remainder is 7. The "sum" variable stores 7. The "num" is then divided by 10 and stored in "num". The loop is executed again and remainder is added to sum. Thus, the total sum is displayed of a number when the while loop exits. Thus, the sum of 4281 is 4 + 2 + 8 + 1 = 15.

2.2.4 Nesting of "While" Loops

Like "for" loops, it is also possible to nest "while" loops as shown in the following example:

```
#Program using nested while loops.
k=1
x=int(input("Input the number of lines: "))

#Using nested while loops.
while k<=x:
    j=1
    while j<=k:
        print(j,end=" ")
        j=j+1
    print("\n")
    k = k+1

#Outside the loop.
print("Finished Task")
```

```
In [1]: runfile(...)
Input the number of lines: 5
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
Finished Task
```

Explanation

This program uses a nested "while" loop for printing the sequence of numbers. The user is prompted to enter the number of lines. If the user enters 5, five lines are printed and each line will display numbers depending on the number of line. For example, the first line will display only number 1; the second line will display numbers 1 and 2; the third line will display numbers 1, 2, and 3, and so on.

2.3 Nesting of Conditional Statements and Loops

It is also possible to nest conditional statement and different loops. For example, conditional statement can be inside for/while loop or for/while loop can be inside conditional statement.

2.3.1 The "for" Loop inside "if" Conditional Statement

The following program shows the usage of for loop inside "if" conditional statement.

```
#Program for printing even/odd numbers depending on user's choice.
choice=int(input("Enter your choice: 1 for odd, 2 for even: "))

#Using for inside if structure.
if choice==1:
    print("Odd numbers in the range are:")
    for k in range(10,21,2):
        print(k)
elif choice==2:
    print("Even numbers in the range are:")
    for k in range(2,11,2):
        print(k)
else:
    print("Invalid choice")

#Outside the loop and decision structure.
print("Finished Task")
```

```
In [1]: runfile(...)
Enter your choice: 1 for odd, 2 for even: 2
Even numbers in the range are:
2
4
6
8
10
Finished Task

In [2]: runfile(...)
Enter your choice: 1 for odd, 2 for even: 1
Odd numbers in the range are:
11
13
15
17
19
Finished Task

In [3]: runfile(...)
Enter your choice: 1 for odd, 2 for even: 5
Invalid choice
Finished Task
```

Explanation
The user is asked to enter the choice: 1 for odd and 2 for even. When the user enters choice as 1, the odd numbers using for ... are printed. If the user enters choice as 2, the first five even numbers using "for" loop are printed. However, the system ... "Invalid choice" if the user enters choice besides 1 or 2.

2.3.2 The "if" Conditional Statement Inside "for" Loop

The following program considers "if" statement inside "for" loop.

```
#Program to compute the length of a given string ignoring spaces.
string=input("Enter the String: ")
count=0

#Use of if inside for loop.
for letter in string:
    if letter !=' ':
        count=count+1

#Outside the loop.
print("The length of the string is: ",count)
print("Finished Task")
```



```
In [1]: runfile(...)
Enter the String: India is my country
The length of the string is: 16
Finished Task
```

Explanation
It can be observed that whenever there is a space, the count variable is not incremented, hence the result consists of only the number of letters in the string.

```
#Program to print all different arrangements of the letters M, N and O.
for first in 'MNO':
    for second in 'MNO':
        if second != first:
            for third in 'MNO':
                if third != first and third != second:
                    print(first + second + third)

#Outside the for loop.
print("Finished Task")
```

```
In [1]: runfile(...)
MNO
MON
NMO
NOM
OMN
ONM
Finished Task
```

Explanation
This program uses a three-level nested "for" loop to print all the different arrangements of the letters A, B, and C. Each string printed is a permutation of ABC. The "if" statement is used to prevent duplicate letters within a given string. Since we need that each letter occurs one time, hence the condition is made in such a way that the letter is printed if first is not equal to third and third is not equal to second. This finally helps to print distinct letters.

QUICK CHALLENGE

Use a nested "for" loop to display the different combinations of letters that can be formed from the input of the five-lettered string given by user.

The following example tries to determine whether one string is kangaroo word of second. Kangaroo refers to a word carrying another word within it but without transposing any letters. For example, encourage contains courage, cog, cur, urge, core, cure, nag, rag, age, nor, rage and enrage but not run, gen, gone, etc. The occurrence of each letter of the second string is checked. If the letter is found in the first string, then the occurrence of the letter is checked after the index of that letter. If the letter is found to be missing at any stage, the flag counter is set to false. If the length of first string is lower than the second string, there is a mismatch. Each letter of the second string is checked in the first string. If the letter occurs in the first string, the next letter is checked from the later index. If any letter is not found, the flag is set to false and result is printed.

```
#Program to determine whether one string is kangaroo word of second.
string1=input("Enter the first string: ")
string2=input("Enter the second string: ")
flag=True

#Check the basic condition.
if len(string1)<len(string2):
    print("Wrong!!!Second string is bigger than the first string")

#Execute if the basic condition is satisfied.
else:
    val=0
    for i in range (0,len(string2)):
        for j in range (val,len(string1)):
            if (string2[i] == string1[j]):
                val = j
                flag=True
                break
            else:
                flag=False
        if flag==False:
            break

#Displaying the output.
if flag==False:
    print("It is not a kangaroo word")
else:
    print("It is a kangaroo word")
```

```
In [1]: runfile(...)
Enter the first string: welcome reader
Enter the second string: lead
It is a kangaroo word

In [2]: runfile(...)
Enter the first string: hello dear
Enter the second string: load
It is not a kangaroo word
```

Explanation

The user is asked to input two strings which are stored in string and `string2`. An if statement is executed to determine whethe first string is smaller than the other string. If it is smaller, this means that the strings are not entered properly. This is obvious that

the second string is contained in first, it should be at least equal to it. The else statement is executed if the strings are properly entered. Two nested for loops are used, since we wanted to match each and every letter of the string (first for loop) with each and every letter of another string (second for loop). Since it is desired that the letter should not be transposed, hence if there is the occurrence of the letter of second string in first string, the second for loop will start from the index of the occurred letter in `string1`. Thus, a variable named "val " is initialized to 0, and when the first search is completed, it will take the value of the index of the first string (*val=j*); this means that the search will continue from the index of last found letter. If any letter is not found in the string, Boolean variable named flag is made False. The last section of the program checks the value of flag. If flag is true, the result of "`It is a kangaroo word`" will be displayed. In the example, when the strings entered are "`welcome reader`" and "`lead`", it will display "`It is a kangaroo word`", because each and every letter of lead is occurring in welcome reader in the same order. In the second example, though each and every letter of load is occurring in "`hello dear`", but the order is not the same, hence it displays that "`It is not a kangaroo word`".

2.3.3 The "if" Conditional Statement Inside "while" Loop

The following program considers "if" statement inside "while" loop.

```
#Program to perform the operation randomly depending on user's choice.
Total = 0
choice="yes"

#Use of if statement inside the while loop.
while choice=="yes":
    operation=input("Choice: A(Add) or S(Sub): ")
    amount = int(input("Enter the number: "))
    if operation=="A":
        Total+=amount
    elif operation=="S":
        Total-=amount
    else:
        pass
    choice=input("Wish to Continue-yes/no: ")

#Outside the loop.
print("The total is: ",Total)
print("Finished Task")
```

```
In [1]: runfile(...)
Choice A(Add) or S(Sub): A
Enter the number: 123
Wish to Continue-yes/no: yes
Choice A(Add) or S(Sub): S
Enter the number: 56
Wish to Continue-yes/no: yes
Choice A(Add) or S(Sub): A
Enter the number: 350
Wish to Continue-yes/no: no
The total is:  417
Finished Task
```

Explanation

Initially the choice is made "yes" and "Total" is initialized to 0. The "while" loop is executed till the user enters "yes". The user can enter the choice of either addition or subtraction. When the user enters "A", the number is added and when the user enters "S", the number is subtracted. In this program, after three times of execution, the user enters "no" and hence the total is printed at last.

2.3.4 Using "for", "while", and "if" Together

The following program uses all "for", "while", and "if" in the same program.

```
#Program to use three control flow statements in a single program.
choice=eval(input("Enter your choice: 1 or 2: "))
if choice==1:
    start=eval(input("Input the starting number of range: "))
    end=eval(input("Input the ending number of range: "))
    for x in range(start,end):
        print(x, end=' ')
elif choice==2:
    ans="y"
    value=111
    while (ans=="y"):
        print(value)
        value=value+1
        ans=input("Press y to continue: ")
else:
    print("Invalid choice")

#Outside the loop and decision making statement.
print("Finished Task")
```

```
In [1]: runfile(...)
Enter your choice: 1 or 2: 1
Input the starting number of range: 15
Input the ending number of range: 20
15 16 17 18 19
Finished Task

In [2]: runfile(...)
Enter your choice: 1 or 2: 2
111
Press y to continue: y
112
Press y to continue: y
113
Press y to continue: n
Finished Task

In [3]: runfile(...)
Enter your choice: 1 or 2: 3
Invalid choice
Finished Task
```

Explanation

This program shows the utility of for, while, and if in the same program. The user is prompted to enter the choice, which is stored i the variable "choice". If the user enters choice as 1, then "for" loop is executed for printing numbers between the specified range by th user. If the user enters 2 for the value of "`choice`", initial value of 111 is stored in the variable "value" and the choice is initialized "y". The program prints the number till the user enters "`ans`" as "`y`" and the value of "`y`" is incremented by 1 in each iteration. T "while" loop is used for this purpose, because the program is not aware how many times the loop will get executed. It should be not that while is ideal for indefinite loops. When the user enters "3" for the value of choice, "`Invalid choice`" is printed.

2.4 Abnormal Loop Termination

We have seen that the loops are executed until the condition is true. This condition is checked only at the top of the loop, hence the loop is not exited if the condition becomes false in the middle of the execution. Generally, this situation does not hap because the loop is created to execute all the statements within the body. But in rare scenarios, if it happens, we need to exi loop by changing execution from its normal sequence. The break, pass, and continue statements in Python give programmers t flexibility for implementing the control logic of loops in these situations. When abnormal loop termination occurs, all objects were created in that scope are destroyed. It should be mentioned here that ideally every loop should have a normal flow and h single entry and exit point. It is discouraged to use abnormal loop termination because it introduces an exception into the n control flow of the loop. However, in some unavoidable situations, these statements can be the only alternative.

2.4.1 Break Statement

Break terminates the loop statement and transfers execution to the statement immediately following the loop. When the "break" statement is encountered inside a loop, the loop is immediately terminated regardless of what iteration the loop may be on and program control resumes at the next statement following the loop. The "break" statement immediately exits a loop, skipping the rest of the loop's body, without checking to see if the condition is true or false. Execution continues with the statement immediately following the body of the loop. It is important to note that the "break" statement exits only the loop in which "break" statement was found.

```
#Program to show use of break statement to terminate for loop in between.
for num in range(11,20):
    print(num)
    if(num>15):
        break

#Outside the loop and decision making statement.
print("Finished Task")
```

```
In [1]: runfile(...)
11
12
13
14
15
16
Finished Task
```

Explanation

Initially, the "for" loop was supposed to execute from num = 11 to num = 20. The intention of the user was that if the value reaches 16, it should exit the "for" loop. The first value (11) was printed and "if" condition was checked. Since the value 11 was not greater than 15, the next value was printed, and so on. When the value 16 was printed and "if" condition was checked, it was found that the value was greater than 15 and hence break statement was executed. This terminates the loop and further values were not printed.

Proper positioning of break statement inside the nested loops is important for the correct execution. Multiple break statements can also be used at different places for effective control.

The break statement can also be used inside a "while" loop also, as demonstrated in the following program:

```
#Program to use break statement to terminate while loop in between.
total = 0
print("Enter positive integer numbers to add; negative number to end.")
while True:
    amount = eval(input("Input the amount: "))
    if amount < 0:
        break
    total += amount

#Outside the loop and decision making statement.
print("The total is =", total)
print("Finished Task")
```

```
In [1]: runfile(...)
Enter positive integer numbers to add; negative number to end.
Input the amount: 40
Input the amount: 90
Input the amount: 80
Input the amount: -9
The total is = 210
Finished Task
```

Explanation

Our intention is to add unlimited positive integers corresponding to the amount of the product; the condition of the "while" can never be false till the user enter positive integers (customers can buy unlimited products). In this scenario, the "break" statement is the only way to exit out of the loop. The "break" statement is executed only when the user enters a negative number. Like "for" loop, the "while" loop is also terminated in between and thus any statem

body are not executed. Thus, when the user enters a negative value, the total is printed. The total is printed after the user enters a negative number, since this statement is outside the while loop.

It should be noted that in a nested loop, the `break` statement exits only the loop in which the break is found. This can be better understood by the following example of `break` statement inside nested loops.

```
#Program to display prime numbers till a number.
limit = eval(input('Enter largest limit for displaying prime numbers:'))
for value in range(2, limit+1):
    is_prime = True
    for factor in range(2, value):
        if (value%factor==0):
            is_prime = False
            break
    if is_prime:
        print(value,end=' ')

#Outside the loop and decision making statement.
print()
print("Finished Task")
```

```
In [1]: runfile(...)
Enter largest limit for displaying prime numbers: 15
2 3 5 7 11 13
Finished Task

In [2]: runfile(...)
Enter largest limit for displaying prime numbers: 30
2 3 5 7 11 13 17 19 23 29
Finished Task
```

Explanation

The first "for" loop tries to determine the prime numbers from 2 (smallest prime number) to limit. It initializes the value of `is_prime = True`. It then divides each number of the value from 2 till the number for determining whether it is a prime number or not. The value of `is_prime` is made "`False`" if the number is divisible by any number (`value%factor==0` implies remainder is 0). The statement "`if is _prime:`" means that if `is_prime` is True, then the value will print, else the new number will be considered. It is important to focus on the spaces used in the program. A change in settings of space will produce a different result.

This program can also be created by using a nested "while" loop with small modifications.

```
#Program to print prime numbers from 5 till the user-specified number.
k=1
limit = int(input("Enter largest limit for printing prime numbers: "))
while (k<limit):
    j=2
    while(j<=(k/j)):
        if not(k%j):
            break
        j=j+1
        if (j>k/j):
            print(k, end=' ')
    k=k+1

#Outside the loop and decision making statement.
print()
print("Finished Task")
```

```
In [1]: runfile(...)
Enter largest limit for printing prime numbers: 25
5 7 11 13 17 19 23
Finished Task

In [2]: runfile(...)
Enter largest limit for printing prime numbers: 60
5 7 11 13 17 19 23 29 31 37 41 43 47 53 59
Finished Task
```

Explanation

This program uses a nested "while" loop for displaying the prime numbers according to the user choice. The program determines the prime number within the range specified by the user. The first "while" loop is used to perform a check related to number and has a counter for increasing the value of number by 1. The condition (`k<limit`) is checked so that the "while" loop is executed till the last number is reached. The nested "while" loop is used for determining whether the number is a prime number. The number is printed only if it is a prime number, else the nested loop is exited using "`break`" statement. The main "while" loop then considers the next number and the whole process is repeated. Thus, it is clear that break exits only the loop where it is found.

It should be noted that using multiple `break` statements within a single loop should be avoided. The user should always try to rewrite the code so that the `break` statement is not used. However, in exceptional cases, it can be used with care.

2.4.2 Continue Statement

During a program's execution, when the `break` statement is found within the loop, the remaining statements within the body of the loop are not executed and the loop is exited. But when a continue statement is found within a loop, the remaining statements within the body of the loop are skipped; however, the loop is executed again and the condition in loop is checked again to see if the loop should continue or be exited. If the loop's condition is still true, the loop is not exited, but the loop's execution continues at the top of the loop. It should be noted that similar to "`break`" in a nested loop, the continue statement affects only the loop in which "`continue`" is found.

```
#Program to add positive integers till user enters 0.
total = 0
complete = False;
while not complete:
    amount = eval(input("Input the number (press 0 to quit): "))
    if (amount<0):
        print("Wrong!!!! You entered a negative value", number)
        continue
    #Skip rest of body for this iteration.
    if (amount==0):
        complete = True
    else:
        total += amount

#Outside the loop and decision making statement.
print("The total number is =", total)
print("Finished Task")
```

```
In [1]: runfile(...)
Input the number (press 0 to quit): 50
Input the number (press 0 to quit): 34
Input the number (press 0 to quit): 70
Input the number (press 0 to quit): 0
The total number is = 154
Finished Task
```

Explanation

The variable "total" is assigned to 0 and "complete" is assigned to False. The "while" loop executes till value of "`complete`" is "`False`". In this program, the "while" loop is used to add numbers till the user enters a number "0". When the user enters negative number, the rest of the loop is not executed and the control transfers to the starting of the loop.

2.4.3 Pass Statement

In programming, `pass` is a null statement. This statement is used when the user has created a code block that it is not required. When `pass` is executed, nothing is executed. This is shown in the following example:

```
#Using pass statement.
for letter in 'HELLO':
    pass
print("The last  letter is: ",letter)
print("Finished Task")
```

```
In [1]: runfile(...)
The last letter is:  O
Finished Task
```

Explanation
In this program, `pass` statement does not let anything happen and hence, only the last letter of word "HELLO" is printed. It helps in just executing the loop again without doing anything.

2.5 Errors and Exception Handling

While writing programs, errors may occur unwillingly which may prevent the program to run according to the requirement of the user. The process of finding and eliminating errors is called *debugging*. Python provides an exception-handling mechanism that enables us to detect errors and handle them efficiently. This enables us to execute program without terminating after an exception is caught. Once the exception is resolved, program execution continues till completion.

2.5.1 Types of Error

The different types of error that can occur in a Python program are compile-time error, run-time errors, and logical errors.

2.5.1.1 Compile-Time Errors

Compile-time errors are syntactical errors found in the code due to which program compilation does not take place. For example, incorrect spelling, incorrect number of parenthesis, not using colon (:) sign after "if", improper indentation, etc. The Python compiler displays error message along with the line number in case of compile-time errors, which helps the user to rectify the error.

```
#Program to show occurrence of compile-time error.
if (x>y):
    print("x is big"
```

```
In [1]: runfile(...)
print("x is big"
                ^
SyntaxError: unexpected EOF while parsing
```

Explanation
In this program, a parenthesis was missing in the `print` statement and hence, a compile-time error displaying the `Syntax Error` was displayed when the program was executed.

2.5.1.2 Run-Time Errors

Run-time errors occur at the time of execution of program. This error occurs after the check has been done for compile-time error, for example, data type mismatch. These errors are however sometimes not understood by the user. Also, in some cases, these are not displayed which leads to abnormal execution and may even lead to endless execution without termination.

```
#Program to show occurrence of run-time error.
number=int(input("Input the number: "))
print("The number is ",number)
```

```
In [1]: runfile(...)
Input the number: 5
The number is 5

In [2]: runfile(...)
Input the number: w
File "<string>", line 1, in <module>
NameError: name 'w' is not defined
```

Explanation

In this program, when the user enters 5, the number is displayed. But in the second case, when the user enters "w", i NameError since int() function only accepts numbers and "w" is a string.

2.5.1.3 *Logical Errors*

Logical errors occur due to the usage of incorrect logic in program by the user. For example, using incorrect condition in " ment to check the data. These logical errors can be corrected by the programmer by modifying the program.

```
#Program to show occurrence of logical error.
choice=int(input("Input the day between 1-7: "))
if (choice==1):
    weekday="Monday"
elif (choice==2):
    weekday="Tuesday"
elif (choice==3):
    weekday="Wednesday"
elif (choice==4):
    weekday="Thursday"
elif (choice==5):
    weekday="Friday"
elif (choice==6):
    weekday="Saturday"
else:
    weekday="Sunday"

#Outside the decision making statement.
print("The day is ",weekday)
```

```
In [1]: runfile(...)
Input the day between 1-7: 1
The day is Monday

In [2]: runfile(...)
Input the day between 1-7: 9
The day is Sunday
```

Explanation

In this program, the user is prompted to enter a number between 1 and 7; the proper result is generated if the user enters a numb between 1 and 7. If the number entered was greater than 6 or less than 1, the result would have been Sunday. In this progra logical errors occur because it displays "Sunday" as the weekday when the number is not falling in the range 1 to 7. In such situation, the user will not be able to understand the reason for error and nature of error.

2.5.2 Exception Handling

Exception handling is an effective tool to handle the errors in the program. The purpose of exception handling is to terminate pro gram properly and display proper message to the programmer in case of an error. If the programmer can determine the type of errors that can occur, then exception is used to do efficient programming and for solving the problem. Some of the common exceptions that are defined in Python include:

1. `ImportError`: Raised when an `import` statement fails.
2. `IndexError`: Raised when a sequence index is out of range.
3. `NameError`: Raised when a local or global name is not found.
4. `IOError`: Raised when an input/output operation fails.
5. `ArithmeticError`: Raised for numerical calculations.
6. `OverflowError`: Raised when a calculation exceeds max limit.
7. `SyntaxError`: Raised when the compiler encounters a syntax error.
8. `ZeroDivisonError`: Raised when there is division by zero.
9. `ValueError`: Raised when there is a mismatch in arguments.

Syntax

```
try:
    --------------------
except Exception I:
    --------------------
except Exception 2:
    --------------------
else:
    --------------------
finally:
    --------------------
```

This program can be created effectively using exception handling as:

```
#Program to show use of exception handling.
choice=int(input("Input the day between 1-7: "))
try:
    if choice<1 or choice>7:
        raise ValueError()
    if (choice==1):
        weekday="Monday"
    elif (choice==2):
        weekday="Tuesday"
    elif (choice==3):
        weekday="Wednesday"
    elif (choice==4):
        weekday="Thursday"
    elif (choice==5):
        weekday="Friday"
    elif (choice==6):
        weekday="Saturday"
    else:
        weekday="Sunday"
    print("The day is ",weekday)
except ValueError:
    print("Wrong!!!!Check week day.....")
```

```
In [1]: runfile(...)
Input the day between 1-7: 3
The day is Wednesday

In [2]: runfile(...)
Input the day between 1-7: 8
Wrong!!!!Check week day.....

In [3]: runfile(...)
Input the day between 1-7: 0
Wrong!!!!Check week day.....
```

Explanation

In this program, when the user enters 3, "`Wednesday`" is printed and when the user enters 8 or 0 as its weekday `Error` is printed. This is because of "`try`" statement which raises an exception from its block if the user enters a value not falling in the range 1–7. The exception is caught in the "while" block and the block of statements inside except clause are executed. Hence, message of `Wrong!!!!....` inside except statement is printed when the user enters 8 or 0.

It is important to note that a single try block can have many except blocks to handle multiple exceptions as shown in the ... However, we can also write a try block without except block but cannot write except block without try block. Besides except ... exception handling has two other important clauses, namely "`finally`" and "else". The else block is executed when no ... occurs, and finally block is always executed at the end of the `try/except/else` block. This is shown in the following ...

```
#Exception handling using multiple except blocks, else and finally.
try:
    newfile=open("firstfile","r")
    value=newfile.readline()
    newvalue=value+100
except ValueError:
    print("Integer is not present")
except IOError:
    print("Error!!!!File is not found")
except:
    print("Some error!!!!")
else:
    print("Success!!!!! Task Completed")
finally:
    print("Close the file")
```

```
In [1]: runfile(...)
Error!!!!File is not found
Close the file

In [2]: runfile(...)
Integer is not present
Close the file

In [3]: runfile(...)
Success!!!!! Task Completed
Close the file
```

Explanation

This program shows multiple except clauses in a single `try` block. When the `try` block returns an Input–Output (IO) exception ... it is caught by `except` block for IO exceptions and statements inside this block are executed. When the `try` block returns a value ... exception, it is caught by "`except`" block for `ValueError` exception and statements inside this block are executed. However ... if any other exception besides the listed two exceptions occurs in the program, it is caught by "`except`" block without any ... exception and statements inside this block are executed. The statements inside the else block are executed if no exception occurs ... Finally, a statement is always executed at last regardless of occurrence of the exception.

In the first case, when the file was not found, the `IOError` was raised and caught by the `IOError()`. The statements ... inside it are printed and then finally the block is executed. Similarly, in the second case, when the file was found, but there was an ... absence of an integer in the file, the `ValueError` exception was raised and hence was caught in the `ValueError` block and ... then finally the block was executed. In the third case, no exception was generated, hence else block was executed and finally the ... block was executed.

Execute different programs considering different types of exception that can occur and handle the situation to solve the problem.

2.6 User-Defined Functions

A function is a set of statements put together to perform a specific task. Functions are very much useful when a block of statements ... has to be written once and executed multiple times with or without different inputs. Thus, the function is a standard unit of reuse. ... Functions are useful when the program size is too large or complex. Functions are called to perform each task sequentially from the

main program. It is like a top-down modular programming technique to solve a problem. Functions are also used to reduce the difficulties during debugging a program by the programmer because of its modularity feature. A function is an object and the interpreter passes control to the function along with arguments for the function to accomplish the task. The function in turn performs its task and returns control to the interpreter as well as any result which is further stored in other objects.

Writing functions is a core activity of a programmer, and Python provides a great support to create user-defined functions. They are created according to the requirement of the user, and once created they are used like the built-in functions. They basically act as a turning point from user to a developer, who creates new functionality for Python which can be shared with others.

The body of function contains a collection of statements that basically define the task of function. When a function is called, we may/may not pass a value to the argument. The user-defined function allows a developer to create an interface to the code, specified with a set of arguments. This interface allows the developer to communicate to the user the aspects of the code that are important or are most relevant. Typically, a function produces a result based on the parameters passed to it. However, it is possible that a function may contain no arguments. Thus, a function has a name, a list of parameters (which may be empty), and a result (which may be None).

Syntax

```
def fname(arg1, arg2, ...):
                    function body
```

where,

- `fname` is the actual name of the function stored in Python environment.
- `arg1, arg2` are the optional arguments. An argument is like a placeholder. When a function is invoked, we pass a value to the argument. This value is referred to as actual parameter or argument. The argument list refers to the order and number of the arguments of a function. A function can/cannot contain arguments.
- `function body` contains a collection of statements that define the task of function.

It should be noted that the function name and the arguments list together constitute the function signature.

The task of the function is defined when a function is created. To perform the defined task, we need to call the function. When a program calls a function, the program control is transferred to the called function. A called function performs a defined task and when its return statement is executed or last statement is reached, it returns the program control back to the main program. To call a function, we simply need to pass the required parameters along with the function name, and if the function returns a value, then we need to store the returned value from a function within a variable.

2.6.1 Function without Arguments

A function without arguments can be created using an empty parentheses, because all arguments inside a function are passed within parentheses. It should be noted that a function without arguments can be created with or without return statement.

```
#Create a function without arguments and without return statement.
def hello():
    print("Welcome to world of Python")
    print("Good Day")

#Calling the function.
hello()
```

```
In [1]: runfile(...)
Welcome to world of Python
Good Day
```

Explanation

The name of the function is `hello` which is succeeded with an empty parenthesis. This means that a function is created without any arguments. The body of the function is written inside the definition of the function. In this program, the function body has two print statements. When the function is executed and called by its name, all the tasks of the function are executed and hence both the print statements are executed and the result is printed accordingly.

The major advantage of function is that it can be created once and called many times. This is illustrated in the following program.

```
#Calling a function multiple times in the same program.
def hello():
    print("Hello")
    print("Enjoy Learning")

#Calling the function multiple times.
hello()
hello()
hello()
```

```
In [1]: runfile(...)
Hello
Enjoy Learning
Hello
Enjoy Learning
Hello
Enjoy Learning
```

Explanation

In this program, the function `hello()` is created once, but it is called three times. Hence both the print statements inside the function are executed three times and hence the output prints six statements.

A function body can also contain some expressions and hence function can also produce some results as explained in the following example:

```
#Creating a function with arguments and compute the result for arguments.
def function1():
    num1=100
    num2=200
    print("The result is: ",num1+num2)

#Calling the function.
function1()
```

```
In [1]: runfile(...)
The result is: 300
```

Explanation

No argument is passed in this function. The function assigns value of 100 to the variable "num1" and 200 to the variable "num2". It then prints addition of "num1" and "num2". When the function is called, it simply returns the result of addition of "num1" and "num2" (100 + 200 = 300).

A function becomes more meaningful and will yield better results if programming is also done using loops and decision-making statements in the function body. This is helpful for doing effective analysis and decision making in Python.

```
#Creating a function without an argument and using a "for" loop.
def function2():
    for num  in range (1,6):
        print(num * 7)

#Calling the function.
function2()
```

```
In [1]: runfile(...)
7
14
21
28
35
```

Explanation

The first statement creates a user-defined function by the name `function2`. There is no argument passed inside the function. The function body starts after the colon and in next indented line. It has for loop which has "num" as the counter variable and a sequence is generated from 1 till 5 (6 - 1). The "for" loop body multiplies each number starting from 1 till 5 with 7 and prints the number. Thus, the table of 7 till 5 is printed.

It is also possible to create multiple functions in the single program as discussed in the following example:

```
#Program for creating multiple functions in a single program.
#Creating area_square function for calculating area of a square.
def area_square():
    side=eval(input("Input the length of square: "))
    print("The area of the square is: ",side*side)

#Creating area_circle function for calculating area of a circle.
def area_circle():
    rad=eval(input("Input the radius of circle: "))
    print("The area of the circle is: ",3.14*rad*rad)

#Creating area_rectangle function for calculating area of a rectangle.
def area_rectangle():
    length,breadth=eval(input("Input length and breadth of rectangle: "))
    print("The area of the rectangle is: ",length*breadth)

#Creating area_triangle function for calculating area of a triangle.
def area_triangle():
    a,b,c=eval(input("Input the three sides of triangle: "))
    s=(a+b+c)/2
    print("The area of the triangle is: ",(((s-a)+(s-b)+(s-c))**0.5)

#Calling the function according to user choice.
choice=int(input("Choice: 1-square, 2-circle, 3-rectangle, 4-triangle: "))
if choice==1:
    area_square()
elif choice==2:
    area_circle()
elif choice==3:
    area_rectangle()
elif choice==4:
    area_triangle()
else:
    print("Not a valid choice")
```

```
In [1]: runfile(...)
Choice: 1-square, 2-circle, 3-rectangle, 4-triangle: 4
Input the three sides of triangle: 4, 5, 6
The area of the triangle is:  2.7386127875258306

In [2]: runfile(...)
Choice: 1-square, 2-circle, 3-rectangle, 4-triangle: 1
Input the length of square: 6
The area of the square is:  36

In [3]: runfile(...)
Choice: 1-square, 2-circle, 3-rectangle, 4-triangle: 2
Input the radius of circle: 5
The area of the circle is:  78.5

In [4]: runfile(...)
Choice: 1-square, 2-circle, 3-rectangle, 4-triangle: 3
Input length and breadth of rectangle: 16, 5
The area of the shape is:  80
```

Explanation

This program creates four functions named "area_triangle", "area_rectangle", "area_circle", and "area_square" with no arguments. All the functions ask for the required variables from the user inside the respective body of function and prints the area of its respective shape. From the main program, the user is asked to give the choice, and depending on the choice entered by the user the respective function is called. Example: area_square() function will be called and executed when the user enters 1, area_circle() function will be called when user enters the value 2 and so on.

It should be noted that the variable created inside the function is not known outside the function. Hence, an error will be generated if the variable is called outside the function. This is demonstrated in the following example:

```
#Program for understanding the scope of a variable in a function.
#Creating function without returning the output.
def trial_func():
    amount = 4000

#Calling the variable defined inside the function.
print(amount)
```

```
In [1]: runfile(...)
NameError: name 'amount' is not defined
```

Explanation

This example creates a function named "trial_func" which does not take any argument and stores 4000 in a variable named "amount". The variable amount cannot be fetched from outside the function. It hence throws an error if the user tries to fetch the value of that variable.

A function can also return some variable which needs to be handled in the main program. We can directly store the returned value from a function to a variable and then print the value of that variable in the main program. The use of return statement will shift the control back to the place from where the function was called. It is important to note that a function can have only one return statement; however, the return statement can return multiple values.

```
#Creating a function without arguments and returning a single value.
def bill():
    print("Welcome")
    amount=4000
    return amount

#Calling a function.
value=bill()
print(value)
```

```
In [1]: runfile(...)
Welcome
4000
```

Explanation

The function named "bill" is created without any argument. The bill() function prints "Welcome" and returns the variable "amount". It is important to understand that since the bill () function returns a variable named "amount", hence when this function is called, the output returned by this function needs to be stored in a variable. Thus, when the function is called, the value of the "amount" which is the returned output from the function is stored in the variable named "value". The answer when printed shows the value 4000, since the value 4000 was returned as an output from the function when it was called.

In Python, the function can also return a Boolean value to the main program unlike most of the programming languages. This is demonstrated in the following program:

```
#Program to create a function that returns a Boolean value.
def multiple():
    num1,num2=eval(input("Input the two numbers: "))
    if num1%num2==0:
        return True
    else:
        return False

#Main program calling the function.
print("The number is a multiple:", multiple())
```

```
In [1]: runfile(...)
Input the two numbers: 40, 5
The number is a multiple:  True

In [2]: runfile(...)
Input the two numbers: 64, 7
The number is a multiple:  False
```

Explanation

This program creates a function named `multiple()` which does not take any argument. The user takes two variables "num1" and "num2". If "num1" is a multiple of "num2", "True" is returned, else "False" is returned. When the function is called for the first time, 40 and 5 are entered as numbers to the program. The expression 40%5 returns the remainder 0, since the number 40 is a multiple of the other number 5; hence, "`True`" is returned. In other example, when 64 is not a multiple of 7, which implies that 64%7 is not equal to 0 which further means that "`False`" is returned.

The user can also create multiple functions in the same program and each function can return output separately from its function. Intentionally, the mentioned example of computing area of shape is taken for explaining the utility of `return` statement.

```
#Program for using return in multiple functions in a single program.
#Creating area_square function for calculating area of a square.
def area_square():
    side=eval(input("Input the length of square: "))
    return side*side

#Creating area_circle function for calculating area of a circle.
def area_circle():
    rad=eval(input("Input the radius of circle: "))
    return 3.14*rad*rad

#Creating area_rectangle function for calculating area of a rectangle.
def area_rectangle():
    length,breadth=eval(input("Input length and breadth of rectangle: "))
    return length*breadth

#Creating area_triangle function for calculating area of a triangle.
def area_triangle():
    a,b,c=eval(input("Input the three sides of triangle: "))
    s=(a+b+c)/2
    return (((s-a)+(s-b)+(s-c))**0.5)
```

```
#Calling the function according to user choice.
choice=int(input("Choice: 1-square, 2-circle, 3-rectangle, 4-triangle: "))
if choice==1:
    area=area_square()
elif choice==2:
    area=area_circle()
elif choice==3:
    area=area_rectangle()
elif choice==4:
    area=area_triangle()
else:
    print("Not a valid choice")
print("The area of the shape is: ",area)
```

```
In [1]: runfile(...)
Choice: 1-square, 2-circle, 3-rectangle, 4-triangle: 2
Input the radius of circle: 8
The area of the shape is:  200.96

In [2]: runfile(...)
Choice: 1-square, 2-circle, 3-rectangle, 4-triangle: 4
Input the three sides of triangle: 5,9,11
The area of the shape is:  3.5355339059327378
```

Explanation

The purpose of this code is same as the earlier example to print the area of the respective shape. But, the methodology is different. In the previous program, the area of the shape was printed from the function body itself. In this program, we use "`return`" statement to pass the control from the function to the main program which returns the value of area of the respective shape. It can be observed that all the functions calculate the area inside the function and return the area of the respective shape to the main program which is further stored in the variable "area" and finally the area is printed during the execution of main program.

Python also supports the return of multiple values as illustrated in the following program. However, it is necessary that the number of values returned from the function should be equal to the number of variables where those values are stored.

```
#Creating a function that returns multiple values.
def amount():
    print("Using return for multiple values")
    Maharashtra = 5000
    Gujarat= 2500
    Delhi = 4000
    return Maharashtra,Gujarat,Delhi

#Calling the function.
a1,a2,a3=amount()
print("Sale in Maharashtra is: ",a1,"Gujarat is: ",a2, "Delhi is: ",a3)
```

```
In [1]: runfile(...)
Using return for multiple values
Sale in Maharashtra is: 5000 Gujarat is: 2500 Delhi is:  4000
```

Explanation

This program creates a function named "`amount`", which returns the sales figures of three states: Maharashtra, Gujarat, and Delhi. Since the function returned three values, hence it is important to store them in three variables, namely `a1`, `a2`, and `a3`. It is also important to note that a difference in the number of variables will result in an error in the program.

2.6.2 Function with Arguments

It has been observed that when the user-defined function is created without any argument, the user is not able to change the input given to the function since the variables are assigned predefined value inside the function. But, in practical scenario, the user needs to pass the input as an argument to the function, because the input will change with every execution and every requirement. It is important to note that number of arguments while creating a function should be equal to number of arguments passed while calling the function. However, we can pass any number of arguments inside the function.

2.6.2.1 *Create a User-Defined Function with Single Argument*

The following example considers only one single argument while creating a function:

```
#Creating a function with single argument for printing table of a number.
def table(num1):
    for i in range(1,11):
        print(num1,"*",i,"=",num1*i)

#Calling the function with 8 as an argument.
table(8)
```

```
In [1]: runfile(...)
8 * 1 = 8
8 * 2 = 16
8 * 3 = 24
8 * 4 = 32
8 * 5 = 40
8 * 6 = 48
8 * 7 = 56
8 * 8 = 64
8 * 9 = 72
8 * 10 = 80
```

Explanation

The "def" statement shows that a user-defined function is created by the name "table". There is only one argument "num1" which is passed inside the function. The function body starts from the next line. It has "for" loop which has "i" as the counter variable and numbers starting from 1 till 11. Hence the last number for the counter "i" will be 10. The "for" loop body multiplies each number that is passed as an argument with "i" and prints the table. Thus, the table of 8 is printed when we call the function and pass the argument as 8.

It is also possible to consider multiple arguments while creating a function as discussed in the following example. This example uses three arguments for creating a table:

```
#Creating a function with multiple arguments.
def newtable(num1,num2,num3):
    for i in range(num2,num3):
        print(num1,"*",i,"=",num1*i)

#Calling a function with three arguments by position of arguments.
print("Printing the table using three arguments")
newtable(7,1,3)

#Calling a function with three arguments by names of the arguments.
print("Printing the table using three names of the arguments")
newtable(num1=7,num2=3,num3=5)

#Calling a function with three arguments by changing order of arguments.
print("Printing the table by changing order of three arguments")
newtable(num3=6, num1=7,num2=4)
```

```
In [1]: runfile(...)
Printing the table using three arguments
7 * 1 = 7
7 * 2 = 14
Printing the table using three names of the arguments
7 * 3 = 21
7 * 4 = 28
Printing the table by changing order of three arguments
7 * 4 = 28
7 * 5 = 35
```

Explanation
The function named "newtable" is created using three arguments: num1, num2, and num3. The num1 argument r
the number for which the table will be printed. The num2 and num3 arguments represent the starting and ending nu
the table. When the function is called for the first time, three arguments are given and since no naming is done, hence t
the position of arguments plays its role and num1 is assigned 7, num2 is assigned 1, and num3 is assigned 3. When the fu
called for the second time, the names of the arguments are also written along with the value, hence the values will be take
ingly. It is important to observe that the order of the variables is not mandatory. Hence, in the third case when the variabl
written in predefined order, the answer is computed accordingly.

It is important that the number of arguments while creating and calling a function should be same, else it generates an e
lustrated in the following example:

```
#Passing unequal number of arguments in the function results into error.
newtable(num1=8)
```

```
In [1]: runfile(...)
TypeError: newfunction() missing 3 required positional arguments: 'num1', 'num2', and 'n
```

Explanation
Since the "newtable" function has three arguments, but in this example, when we call the function with one argumen
is generated. This means that passing unequal number of arguments results in an error in this example.

It is possible to create multiple functions in the same program having different numbers of arguments. For proper understan
same "area" program is considered intentionally with different number of arguments in each function and without return s

```
#Program to compute area with arguments and without return statement.
#Creating area_square function for calculating area of a square.
def area_square(side):
    print("The area of the square is: ",side*side)

#Creating area_circle function for calculating area of a circle.
def area_circle(rad):
    print("The area of the circle is: ",3.14*rad*rad)

#Creating area_rectangle function for calculating area of a rectangle.
def area_rectangle(length,breadth):
    print("The area of the rectangle is: ",length*breadth)

#Creating area_triangle function for calculating area of a triangle.
def area_triangle(a,b,c):
    s=(a+b+c)/2
    print("The area of the triangle is: ",(((s-a)+(s-b)+(s-c))**0.5))

#Calling the function according to user choice.
choice=int(input("Choice: 1-square, 2-circle, 3-rectangle, 4-triangle: "))
if choice==1:
    side=eval(input("Input the length of square: "))
    area_square(side)
elif choice==2:
    rad=eval(input("Input the radius of circle: "))
    area_circle(rad)
elif choice==3:
    length,breadth=eval(input("Input length and breadth of rectangle: "))
    area_rectangle(length,breadth)
elif choice==4:
    a,b,c=eval(input("Input the three sides of triangle: "))
    area_triangle(a,b,c)
else:
    print("Not a valid choice")
```

```
In [1]: runfile(...)
Choice: 1-square, 2-circle, 3-rectangle, 4-triangle: 3
Input length and breadth of rectangle: 5, 6
The area of the rectangle is:  30
```

Explanation

It can be observed that since all the functions for computing area are defined with arguments, hence it is important to pass the arguments when these functions are called. Thus, in the main program the user is prompted to enter the values of the respective shape and these values are passed as an argument to the function when the function is called. The function itself does not take input from the user and just does the computation and prints the result. It should be clear that all the programs for computing the area do the same task and hence the result is absolutely the same, but the programming becomes different because of different signature of functions.

The user is prompted to enter the choice from 1 to 4 for the shapes. When the user enters 3, the user is asked to give input of length and breadth of rectangle; the `area_rectangle()` function is called after passing the arguments length and breadth. The `area_rectangle()` function does the computation inside it and prints the result from its function body itself. After the last line of the `area_rectangle()` function is executed, the control is returned back to the program.

We will consider the same "area" program for understanding the difference in programming when the function is created with arguments and also returns the value to the main program.

```
#Creating functions with arguments and with return statement.
#Creating area_square function for calculating area of a square.
def area_square(side):
    return side*side

#Creating area_circle function for calculating area of a circle.
def area_circle(rad):
    return 3.14*rad*rad

#Creating area_rectangle function for calculating area of a rectangle.
def area_rectangle(length,breadth):
    return length*breadth

#Creating area_triangle function for calculating area of a triangle.
def area_triangle(a,b,c):
    s=(a+b+c)/2
    return (((s-a)+(s-b)+(s-c))**0.5)

#Calling the function according to user choice.
choice=int(input("Choice: 1-square, 2-circle, 3-rectangle, 4-triangle: "))
if choice==1:
    side=eval(input("Input the length of square: "))
    area=area_square(side)
elif choice==2:
    rad=eval(input("Input the radius of circle: "))
    area=area_circle(rad)
elif choice==3:
    length,breadth=eval(input("Input length and breadth of rectangle: "))
    area=area_rectangle(length,breadth)
elif choice==4:
    a,b,c=eval(input("Input the three sides of triangle: "))
    area=area_triangle(a,b,c)
else:
    print("Not a valid choice")
print("The area of the shape is: ",area)
```

```
In [1]: runfile(...)
Choice: 1-square, 2-circle, 3-rectangle, 4-triangle: 2
Input the radius of circle: 6
The area of the shape is:  113.03999999999999
```